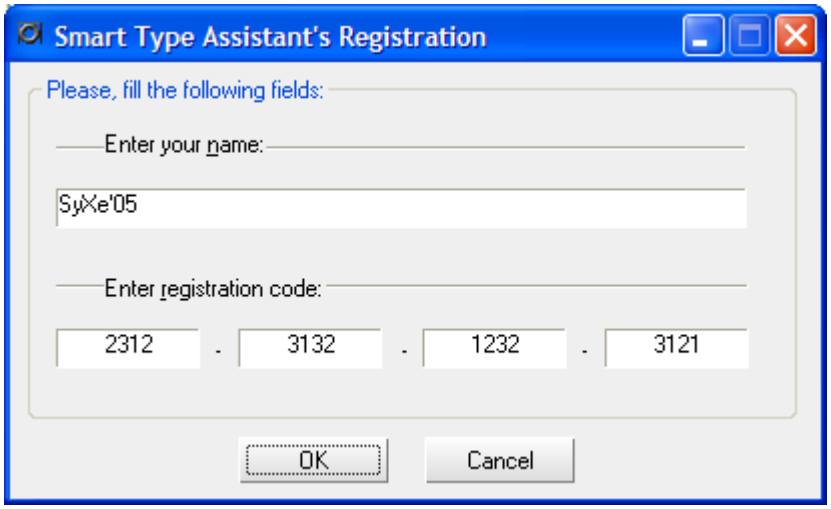
Programa:	Smart Type Assistant 1.3.1.0	
Protección:	Cinderella + Antidebug que no superaremos ;-(	
Objetivos:	Encontrar un serial válido	
URL descarga:	www.blazingtools.com	
Dificultad:	Para mi es difícil el antidebug!	
Herramientas:	OllyDbg v1.10	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 09-09-2005

Hola a todos!!!

Vamos a ver una protección de las más sencillas que nos podemos encontrar, un ejecutable sin comprimir/encriptar con un sencillo algoritmo de validación de serial, no es habitual esto último pues suelen ser bastante enrevesaos y complejos lo que hace difícil seguirlos a veces, pero com odigo no es el caso que nos ocupa, el programa en cuestión hemos visto que se trata de Smart Type Assistant en su versión (Build) 1.3.1.0 de la compañía Blazing Tools, según vimos en el Perfect KeyLogger de esta misma compañía el trabajo no será muy complicado.

Al abordaje...

Parecemos piratas ¿¿?? Jaja.. cuando ejecutamos el programa no vemos ninguna pantalla ni nada, lo que ocurre es que se queda en la barra de tareas minimizado, hacemos doble clic allí y aparece en pantalla, buscamos la opción de registro y no la vemos, la encontramos al final en el iconito de la barra de tareas que hemos picado haciendo un clic con el botón derecho sobre él, así desplegará un menú donde sí encontramos "Enter registration code…" picamos allí pero ya y sale el cuadro de registro, es este:



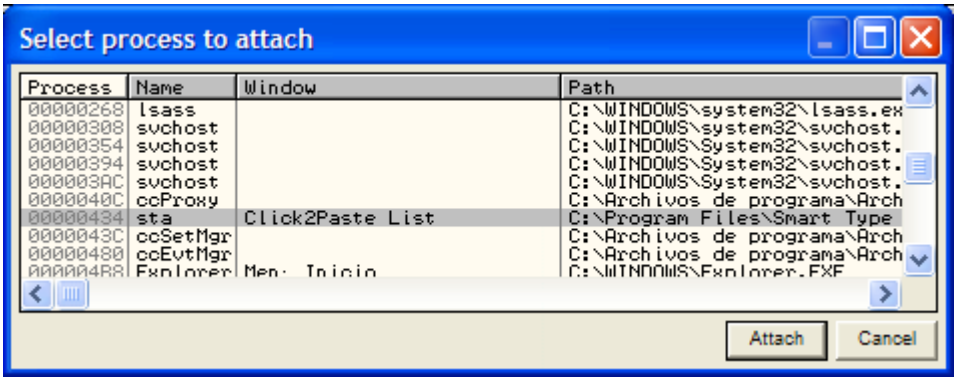
Ya lo hemos rellenado y pulsado "OK" aparece el típico mensaje de error siciendo que si nos hemos equivocado, bla, bla.., ni caso, vale lo cerramos y lo abrimos con Ollydbg, de entrada damos RUN para que corra y se ejecute normalmente, abrimos el cuadro de registro otra vez y ponemos unos datos, ahora antes de pulsar OK vamos a poner algunos BP's concretamente:

BP MessageBoxA/W

Por si las moscas los dos, veremos si salta (todavía no lo he probado yo tampoco), puestos los BP's picamos en OK y ¿el programa se cierra? Jodeeeeeeer no ha sacado mensaje ni nada y encima se cierra, nos ha pillado los brekpoints? Repitamos el proceso a ver si fue algún error externo que justo ha coincidido, no, no es ningún error parece que detecta los BP's normales, pues nada cambiemos a HE'S, jeje.., pongamos pues:

HE MessageBoxA/W

Que son BPHardware's a ver si los pilla ahora hombre, pues también se cierra ¿¿?? Un poco desconcertante es la verdad, pongámos los BP's en otra API, a ver en GetWindowTextA y por si acaso en GetDlgItemTextA y los ponemos HE, espera, espera, ahora no he puesto ningún BP y también se ha cerrado, pienso que se da cuenta que está siendo depurado por eso se cierra, probamos fuera de Olly y el mensaje de error aparece, probemos de attachearlo:



pulsamos Attach y queda parado en ntdll.DbgBreakPoint, pulsamos F9 y luego F12 para pausar el programa, intentamos registrarnos y ni con esas, el programa se cierra!!

Desconcertados del todo buscamos ayuda externa, incluso cambiando de Olly (parcheado, cin parchear, con el Debugger ocultado…) el programita no se deja depurar, encontramos en wdasm un perfecto aliado porque accedemos al corazón del programa:

Encontramos todas las cadenas interesantes, incluso encontramos el _strCmp donde compara el serial válido con el no válido:

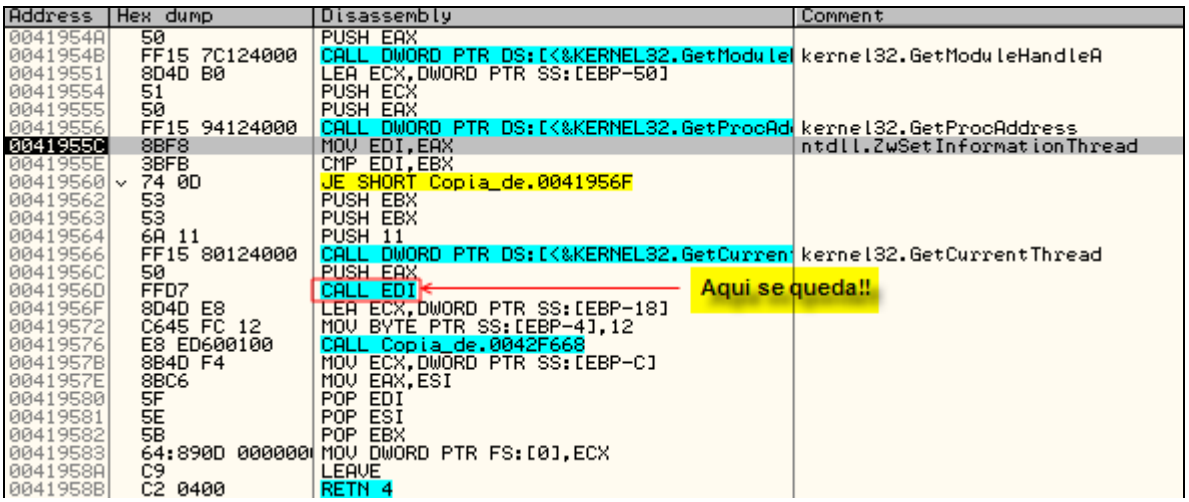
```
:00417B24 8D459C                  lea eax, dword ptr [ebp-64]
:00417B27 50                      push eax
:00417B28 8D8534FFFFFF            lea eax, dword ptr [ebp+FFFFFF34]
:00417B2E 50                      push eax

* Reference To: KERNEL32.lstrcmpiA, Ord:0395h
                                  |
:00417B2F FF159C124000            Call dword ptr [0040129C]
:00417B35 85C0                    test eax, eax
:00417B37 6A40                    push 00000040
:00417B39 7528                    jne 00417B63

* Possible StringData Ref from Data Obj ->"Registration"
                                  |
:00417B3B 68F4DB4000              push 0040DBF4

* Possible StringData Ref from Data Obj ->"Registration succeeded. Thank "
                                       ->"you for choosing Smart Type Assistant!"
                                  |
:00417B40 68ACDB4000              push 0040DBAC
:00417B45 57                      push edi
```

Vemos como carga en EAX ambos seriales en :417B24 y :417B2E y se los pasa a la fución que los compara lstrcmpA como argumento, ésta retornará EAX==0 si son iguales o -1 si no lo son, de ahí el posterior salto JNE saltará si EAX=!0, el problema es que no podemos acceder a la zona "en caliente" por alguna técnica antidebug probablemente (no creo que sea fallo de Olly claro), traceando el código desde el inicio encontramos esta zona:



Llama a GetProcAddress donde obtiene la dirección de ZwSetInformationThread de la ntdll.dll, después el CALL EDI hace una llamada a esta función, si la queremos superar con F8 adiós el programa se jode de mala manera, parece que se descarga de la memoria y todo ¿? Un comportamiento espantoso! He buscado la API en todas las ayudas y no la he encontrado, me queda darme una vuelta por Internet y conseguir información de la misma porque parece ser que tiene mucho que ver en esto

Como no tengo mucho tiempo prometo que miraré este tema (pues ya son muchos los trucos antidebug que me he encontrado) y daré una explicación si la encuentro a este tema, pero antes daré una solución bien sencilla para ver el serial válido, jejeje…

El problema está en que el programa no deja tracear ni una línea de este código, qeuizá tenga algo que ver con VirtualProtect y/o VirtualQuery ¿? Pero sinceramente lo dudo después de lo que hemos visto, además aún dándole Acceso total (Set Acces->Full Access) desde la ventana de memoria a esta sección seguimos igual. Tenemos la opción del parcheo pero no creo que haga falta. La solución que pensé es esta: No podemos tracear ni una instrucción, vale, además no podemos poner BP's en la zona y dar RUN porque igual se cuelga, parece como si se borrara el código ¿? También vale, ahora lo que sí podemos es Attachearlo una vez está en ejecución verdad? entonces haremos lo siguiente: Abrimos un Olly nuevo y cargamos el archivo "sta.exe" que es el ejecutable a estudiar, como tenemos la dirección exacta de donde se verifica el serial gracias a W32DASM vamos hasta allí y ensamblamos un loop infinito en la llamada a la API 'lstrcmpiA' y salvamos los cambios como "sta_loop.exe"

Address	Hex dump	Disassembly	Comment
00417B24	8D45 9C	LEA EAX,DWORD PTR SS:[EBP-64]	
00417B27	50	PUSH EAX	
00417B28	8D85 34FFFFFF	LEA EAX,DWORD PTR SS:[EBP-CC]	
00417B2E	50	PUSH EAX	
00417B2F -	EB FE	JMP SHORT sta.00417B2F	Loop inifinito..!!
00417B31	9C	PUSHFD	
00417B32	1240 00	ADC AL,BYTE PTR DS:[EAX]	
00417B35	85C0	TEST EAX,EAX	
00417B37	6A 40	PUSH 40	
00417B39 v	75 28	JNZ SHORT sta.00417B63	
00417B3B	68 F4DB4000	PUSH sta.0040DBF4	ASCII "Registration"
00417B40	68 ACDB4000	PUSH sta.0040DBAC	ASCII "Registration succeeded.
00417B45	57	PUSH EDI	
00417B46	FF15 44144000	CALL DWORD PTR DS:[<&USER32.MessageBoxA	USER32.MessageBoxA
00417B4C	8D45 9C	LEA EAX,DWORD PTR SS:[EBP-64]	

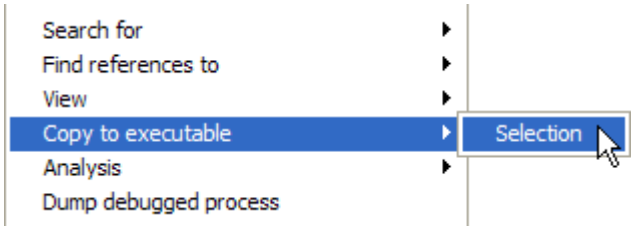
Creamos el loop justo en el CALL que iba a 'lstrcmpiA' que como todos sabemos se utiliza para comparar cadenas entre sí, si son iguales el retorno será EAX==0, bien para esto cambiamos:

Address	Hex dump	ASCII
00417B2F	FF 15 9C 12 40 00 85 C0 6A 40 75 28 68 F4 DB 40	¤£‡@.à└j@u(h¶■@
00417B3F	00 68 AC DB 40 00 57 FF 15 44 14 40 00 8D 45 9C	.h¼■@.W §D¶@.ìE£
00417B4F	50 8D 85 68 FF FF FF 50 E8 9A FE FF FF 59 59 6A	Pìàh PÞü■ YYj
00417B5F	01 58 EB 1E 68 98 DB 40 00 68 44 DB 40 00 57 FF	☺Xû▲hÿ■@.hD■@.W

Estos son los bytes originales del CALL que cambiaremos por estos otros:

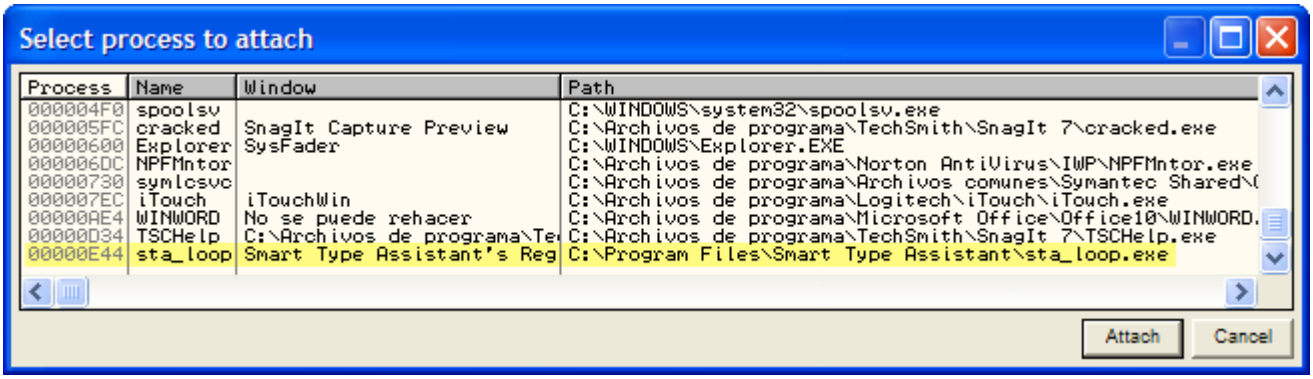
Address	Hex dump	ASCII
00417B2F	EB FE 9C 12 40 00 85 C0 6A 40 75 28 68 F4 DB 40	ù■£‡@.à└j@u(h¶■@
00417B3F	00 68 AC DB 40 00 57 FF 15 44 14 40 00 8D 45 9C	.h¼■@.W §D¶@.ìE£
00417B4F	50 8D 85 68 FF FF FF 50 E8 9A FE FF FF 59 59 6A	Pìàh PÞü■ YYj
00417B5F	01 58 EB 1E 68 98 DB 40 00 68 44 DB 40 00 57 FF	☺Xû▲hÿ■@.hD■@.W

Y ya hemos creado el bucle infinito en la instrucción que queremos, ahora salvamos los cambios y nombramos el archivo como 'sta_loop.exe'

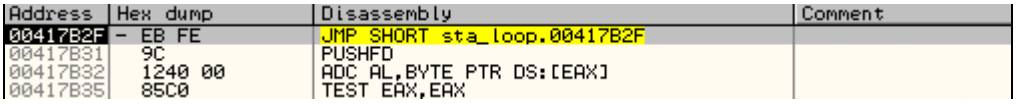


Esto lo hacemos con los bytes seleccionados (los que modificamos nosotros) después se abrirá otra ventana donde elegiremos 'Save file' y le damos el nombre que queramos, yo le di el que dije arriba y con esto terminamos este paso, sigamos con el siguiente…

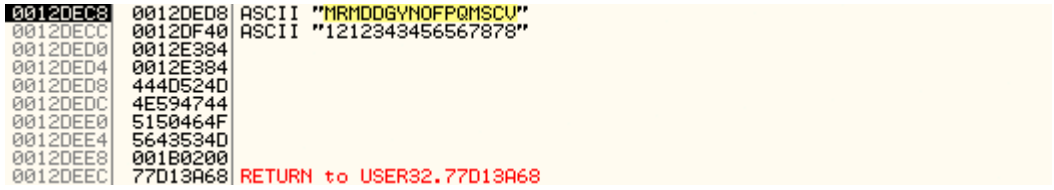
Una vez tenemos el archivo salvado (el del loop) lo ejecutamos, éste se abrirá normalmente, en el cuadro de registro introducimos nuestro nombre, un serial cualquiera y aceptamos, en este momento quedará loopeando en la instrucción modificada y estará a nuestra merced, jeje.. lo que hacemos es attachearlo desde Olly en este momento:



Quedará como siempre detenido en ntdll.DbgBreakPoint, pulsamos F9 y luego F12 para pausar el programa, en este momento estamos justo en el loop infinito que creamos antes:

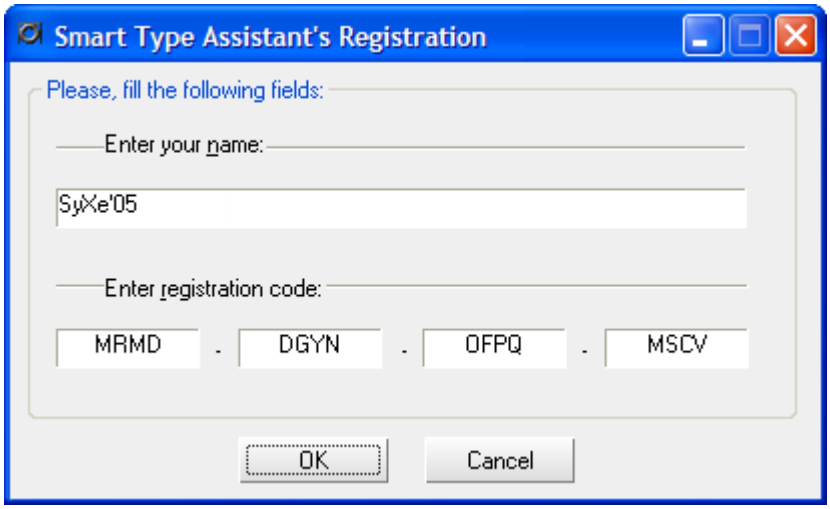


Allí estamos detenidos sobre el salto, como vemos quedó allí saltando dobre la misma instrucción, jeje.. lo que nos dio tiempo a attachear y ahora miremos al stack:

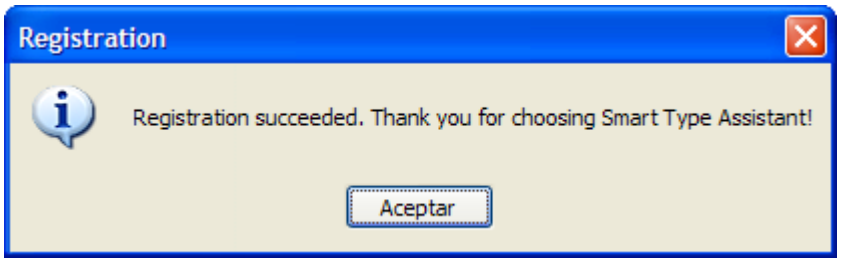


Que buena pinta tiene ese código que aparece al lado del nuestro, está clarísimo que es nuestro serial, este se generó seguramente para nuestro nombre de usuario, para cada uno se generará diferente imagino, la mayoría de programas lo hacen así.

Ahora que tenemos lo que buscábamos sólo nos queda cerrar nuestro trabajo con un buen broche, cerramos este Olly porque si intentamos tracear o incluso pulsar F9 se cerrará igual, mucho antidebug pero de poco le va a servir al amigo, probemos el serial va!



Recordad que el nombre que se debe poner es el mismo que pusimos anteriormente para evitar que se genere uno distinto y nos diga eso de "invalid zerial, sorry, bla, bla.." llegados a este punto nos queda nada más pulsar OK y…



Estaba claro no? No hay que ser muy listo para saber que era el correcto con sólo saber de donde hemos rescatado el seial (del stack en plena rutina de comprobación) junto a nuestro serial y ante la llamada a lstrcmpiA demasiados ingredientes como para no terminar en un final feliz.

Otro día continuamos estudiando protecciones, de esta nos quedamos con el antidebug que no hemos superado, hemos hecho 'trampa' para obtener el serial ;-))

Para cualquier tema me encontraréis aquí:

syxe00@yahoo.es
syxe00@hotmail.com